PLATFORMA REZERWACJI NOCLEGÓW · POLSKA

StayMap Polska

Dokumentacja Biznesowa Projektu

DOKUMENTACJA BIZNESOWA PROJEKTU

v2.0 · Kwiecień 2026

STATUS	ZAKRES	DATA OPRACOWANIA
Projekt zrealizowany	**12 modułów Django**	**Kwiecień 2026**
Gotowy do wdrożenia	44 widoki Next.js	Wersja dokumentu 2.0

KLUCZOWE WYREŃNIKI PLATFORMY

Map-first UX	Wyszukiwanie na mapie Leaflet z klasteryzacją markerów
Dynamiczny cennik	Sezonowość, polskie święta, dopłaty za gości, rabaty za długi pobyt
Asystent AI	Wyszukiwanie w języku naturalnym po polsku — OpenAI / Groq
Czat WebSocket	Komunikacja w czasie rzeczywistym między gościem a gospodarzem
Blind Release recenzji	Obie strony oceniają niezależnie przed upublicznieniem wyników
9 trybów podróży	Romantic, family, pet, workation, slow, outdoor, lake, mountains, wellness
Inteligencja lokalizacji	Automatyczny POI scoring z OpenStreetMap / Overpass API
Porównywarka ofert	Do 3 ofert side-by-side dla anonimowych i zalogowanych użytkowników

StayMap Polska · Dokumentacja Biznesowa · Wersja 2.0 · Kwiecień 2026

Spis treści

1.	**Streszczenie wykonawcze**	**3**
2.	**Cel produktu i grupy użytkowników**	**5**
3.	**Jak działa aplikacja — scenariusze użycia**	**6**
3.1	Scenariusze gościa (A, B, C)	6
3.2	Scenariusze gospodarza (D, E)	7
3.3	Scenariusz administratora (F)	8
4.	**Architektura systemu**	**9**
5.	**Stack technologiczny**	**11**
6.	**Moduły domenowe backendu**	**14**
7.	**Funkcjonalności — opis szczegółowy**	**17**
7.1	Konta i uwierzytelnianie użytkowników	17
7.2	Oferty i wyszukiwanie	18
7.3	Silnik cen i rezerwacje	19
7.4	Panel gospodarza i onboarding	21
7.5	Moderacja treści	22
7.6	System recenzji — blind release	22
7.7	Komunikacja i czat WebSocket	23
7.8	Asystent AI — wyszukiwanie naturalne	24
7.9	Odkrywanie ofert i porównywarka	25
7.10	Inteligencja lokalizacji (POI & scoring)	25
8.	**Frontend — mapa aplikacji i widoki**	**27**
9.	**API — przegląd endpointów**	**31**
10.	**Bezpieczeństwo systemu**	**33**
11.	**Operacje i wdrożenie produkcyjne**	**34**
12.	**Testy i CI/CD**	**36**
13.	**Zależności zewnętrzne**	**37**
14.	**Diagramy przepływów**	**38**
15.	**Spełnienie minimalnych wymagań projektu**	**42**
16.	**Otwarte tematy i dalszy rozwój**	**46**
17.	**Planowane funkcjonalności — Roadmap**	**48**
17.1	AI: Kiedy jechać? — optymalny termin	48
17.2	W X godzin — wyszukiwanie izochron	49
17.3	Pamiętnik z podróży — zdjęcia gości	50
17.4	Karta podarunkowa — voucher prezentowy	50
17.5	Mapa wspomnień — travel journal	51
18.	**StayMap Polska — szanse na sukces**	**52**
18.1	Luka rynkowa — dlaczego teraz i Polska	52
18.2	Wyróżniki technologiczne	53
18.3	Model biznesowy i skalowalność	54
18.4	Strategia wejścia na rynek	55
18.5	Dlaczego StayMap wygra	55

19. **Słowniczek pojęć** **57**

1. Streszczenie wykonawcze

StayMap Polska to kompletna platforma rezerwacji noclegów turystycznych w Polsce, zbudowana z naciskiem na wygodę użytkownika, przejrzystość cen i nowoczesny UX. Centralnym elementem aplikacji jest interaktywna mapa — gość odkrywa, porównuje i rezerwuje noclegi w jednym spójnym interfejsie.

MODUŁY BACKEND	WIDOKI FRONTEND	TESTY BACKEND	TRYBY PODRÓŻY
12	**44**	**25**	**9**
aplikacji Django	strony Next.js	plikiów testowych	profili wyszukiwania

Kluczowe wskaźniki projektu

Obszar	Opis
Docelowy rynek	Polska — turystyka krajowa (góry, morze, jeziora, wellness, agroturystyka)
Waluta	Złoty polski (PLN)
Grupy użytkowników	Goście (podróżni), Gospodarze (właściciele obiektów), Administratorzy
Status projektu	Zrealizowany — wszystkie funkcjonalności i widoki gotowe, CI/CD skonfigurowane
Gotowość produkcyjna	Backend i frontend gotowe do wdrożenia; Stripe przygotowany infrastrukturalnie
Liczba modułów backend	12 aplikacji Django: users, listings, bookings, pricing, search, host, moderation, reviews, messaging, ai_assistant, discovery, location_intelligence
Liczba widoków frontend	44 strony Next.js w trzech grupach tras: (auth), (main), (host)

Kluczowe wyróżniki platformy

- Map-first UX — wyszukiwanie na mapie Leaflet z klasteryzacją markerów jako główny interfejs odkrywania noclegów
- Silnik cen uwzględniający sezonowość, polskie święta ustawowe (GUS), niestandardowe szczyty turystyczne, dopłaty za gości i rabaty za długi pobyt
- Asystent AI rozumiejący zapytania w języku naturalnym po polsku — np. 'domek dla 4 osób na Mazurach z sauną do 300 zł/noc'
- Czat w czasie rzeczywistym między gościem a gospodarzem (WebSocket, Django Channels)
- Moderacja zapewniająca jakość treści przed publikacją — każda oferta wymaga zatwierdzenia administratora
- Mechanizm ślepego ujawnienia recenzji — obie strony oceniają niezależnie przed upublicznieniem
- Inteligencja lokalizacji — automatyczne podsumowanie POI w okolicy każdej oferty (OpenStreetMap / Overpass API)
- Porównywarka ofert działająca dla użytkowników anonimowych i zalogowanych
- Trymby podróży z personalizowanym scoringiem — 9 profili: romantic, family, pet, workation, slow, outdoor, lake, mountains, wellness

2. Cel produktu i grupy użytkowników

Platforma odpowiada na konkretną lukę rynkową: brak w Polsce dedykowanego narzędzia do odkrywania noclegów turystycznych z mapą jako głównym interfejsem, dynamicznym cennikiem uwzględniającym polski kalendarz oraz wbudowaną komunikacją. StayMap Polska integruje perspektywę trzech grup użytkowników w jednym systemie.

Grupa	Potrzeby	Obsługa w systemie
Gość (podróżny)	Znalezienie odpowiedniego noclegu, sprawdzenie ceny i dostępności, szybka rezerwacja, zarządzanie pobytami, komunikacja z gospodarzem	Wyszukiwanie na mapie i filtry, karta oferty, widget rezerwacyjny, konto z historią, lista życzeń, czat, recenzje, porównywarka, asystent AI
Gospodarz (właściciel)	Publikacja i promocja oferty, zarządzanie kalendarzem i cennikiem, obsługa gości, kontrola jakości rezerwacji	Panel hosta, onboarding, CRUD ofert + upload zdjęć, reguły cenowe, kalendarz blokad, kolejka rezerwacji, szablony wiadomości, recenzje
Administrator (moderator)	Zapewnienie jakości i bezpieczeństwa treści, weryfikacja ofert przed publikacją	Kolejka PENDING, akcje zatwierdź/odrzuć z komentarzem, panel administracyjny Django Admin z rozbudowanymi widokami

Założenia produktowe

System skupia się wyłącznie na Polsce i polskim rynku noclegowym. Przekłada się to na konkretne decyzje projektowe:

- Polska lokalizacja interfejsu (język: pl, strefa czasowa: Europe/Warsaw)
- Obsługa polskich świąt ustawowych w silniku cen (zgodnie z przepisami GUS, zaszyte w kodzie bez potrzeby konfiguracji)
- Geokodowanie w oparciu o OpenStreetMap (Nominatim) — adresy i miejscowości polskie
- Dane POI z Overpass API z naciskiem na atrakcje turystyczne charakterystyczne dla polskich regionów
- Waluta rozliczeniowa: PLN. Cennik uwzględnia szczyty specyficzne dla rynku polskiego: wakacje letnie, ferie zimowe, długie weekendy majowe, Boże Narodzenie, Nowy Rok
- Administratorzy mogą aktywować dodatkowe szczyty: Andrzejki (30 lis), Dzień Kobiet (8 mar), Wszystkich Świętych (1 lis)

3. Jak działa aplikacja — scenariusze użycia

Poniżej sześć kluczowych scenariuszy pokazujących jak platforma działa z perspektywy każdej z ról: gościa, gospodarza i administratora.

3.1 Scenariusze gościa

Scenariusz A [GOŚĆ] **Odkrycie i rezerwacja noclegu**	
Gość	Wchodzi na stronę główną i wpisuje w wyszukiwarce: Mazury, 3–7 sierpnia, 2 osoby
System	Geokoduje lokalizację przez Nominatim → wyświetla mapę Leaflet z markerami ofert
Gość	Wybiera tryb podróży lake → TravelModeRanker przelicza scoring, wyniki sortują się
Gość	Klika ofertę na mapie → otwiera kartę ze zdjęciami, opisem, POI i widżetem rezerwacji
Gość	Wybiera daty → system wywołuje POST /bookings/quote/ i pokazuje rozbicie ceny na żywo
Gość	Klika Zarezerwuj → tworzona rezerwacja PENDING → AWAITING_PAYMENT
System	Celery wysyła e-mail potwierdzający do gościa i gospodarza
Gość	Opłaca rezerwację → status zmienia się na CONFIRMED

Scenariusz B [GOŚĆ] **Wyszukiwanie przez Asystenta AI**	
Gość	Na stronie /ai wpisuje: Spokojny domek z sauną dla 4 osób, do 300 zł/noc, sierpień
AI	Pierwszy call LLM (gpt-4o-mini) → parsuje: tryb=slow+wellness, goście=4, amenity=sauna, max_price=300
System	SearchOrchestrator uruchamia geo-filtrowanie + scoring trybów (zawsze świeże dane)
AI	Drugi call LLM → generuje 2–3 zdaniowe wyjaśnienie dla każdej oferty po polsku
Gość	Widzi karty: Ten domek spełnia Twoje oczekiwania, ponieważ posiada saunę...
Gość	Zadaje pytanie uzupełniające: A czy są opcje z basenem? → follow-up w tej samej sesji AI

Scenariusz C [GOŚĆ] **Porównanie ofert i recenzja po pobycie**	
Gość	Na karcie oferty klika Dodaj do porównania (maks. 3 oferty, sesja 48h)
System	CompareSession przechowuje oferty — działa dla anonimowych i zalogowanych użytkowników
Gość	Na /compare widzi tabelę side-by-side: cena, lokalizacja, amenities, oceny
Po pobycie	Celery wysyła e-mail 7 dni po wymeldowaniu z prośbą o recenzję
Gość	Wystawia ocenę (1–5 gwiazdek) + opcjonalne subskory: czystość, lokalizacja, komunikacja
System	Recenzja is_public=False — czeka na recenzję gospodarza (mechanizm blind release)
System	Gdy obie strony ocenią → atomowe ujawnienie obu recenzji jednocześnie

3.2 Scenariusze gospodarza

Scenariusz D [GOSPODARZ] **Publikacja pierwszej oferty**	
Gospodarz	Rejestruje się → przechodzi onboarding (POST /host/onboarding/start/) → uzyskuje rolę hosta

Gospodarz	Na /host/new-listing tworzy ofertę: tytuł, opis, lokalizacja GPS, typ noclegu, tagi geo
Gospodarz	Dodaje zdjęcia (upload z walidacją MIME przez Pillow + python-magic)
Gospodarz	Konfiguruje cennik: cena bazowa, sezonowe mnożniki, rabat za długi pobyt
Gospodarz	Klika Wyślij do moderacji → status zmienia się z DRAFT na PENDING
Admin	Weryfikuje ofertę w kolejce i zatwierdza → status APPROVED
System	Oferta pojawia się w wynikach wyszukiwania i na mapie

Scenariusz E [GOSPODARZ] **Obsługa rezerwacji i komunikacja z gościem**	
System	Gość wysłał prośbę (tryb REQUEST) → gospodarz otrzymuje powiadomienie w panelu
Gospodarz	Na /host/bookings/pending widzi prośbę z datami, liczbą gości i cenowym breakdown
Gospodarz	Ma okno 24h na decyzję; klika Akceptuj → status PENDING → AWAITING_PAYMENT
Gość	Opłaca → CONFIRMED; gospodarz widzi rezerwację w kalendarzu
Gospodarz	Otwiera czat na /host/messages; wysyła wiadomość powitalną z szablonu {{guest_name}}
Czat	Wiadomości przez WebSocket — gość widzi je natychmiastowo bez odświeżania strony
Gospodarz	Po wymeldowaniu wystawia recenzję gościa → mechanizm blind release aktywuje się

3.3 Scenariusz administratora

Scenariusz F [ADMIN] **Moderacja ofert i zarządzanie platformą**	
Admin	Loguje się — konto ma flagę is_admin=True; widzi w panelu kolejkę PENDING
Admin	Na /admin/moderation/listings/ otwiera ofertę, sprawdza zdjęcia, opis, lokalizację
Admin	Zatwierdza → APPROVED lub odrzuca z komentarzem → REJECTED; host jest powiadamiany
Admin	W Django Admin zarządza kolekcjami kuratorowanymi (DiscoveryCollection) dla strony głównej
Admin	Może aktywować dodatkowe szczyty cenowe (Andrzejki, Dzień Kobiet) dla całej platformy
System	Celery Beat co miesiąc generuje raport kosztów OpenAI API — admin monitoruje wydatki

4. Architektura systemu

3.1 Widok ogólny — warstwy systemu

System jest zorganizowany w cztery logiczne warstwy, każda z jasno określoną odpowiedzialnością:

Warstwa	Technologia	Odpowiedzialność
Warstwa prezentacji	Next.js 14 (React 18, TypeScript)	SSR, CSR i ISR stron, App Router, BFF proxy dla Django API
Warstwa API	Django REST Framework + Daphne ASGI	Endpointy REST, WebSocket, autoryzacja JWT, throttling
Warstwa logiki	Django services + Celery + Channels	Logika biznesowa, zadania asynchroniczne, broadcasting WebSocket
Warstwa danych	PostgreSQL 16 + PostGIS 3.4 + Redis 7	Persystencja, zapytania geograficzne (GiST), cache, broker

3.2 Diagram zależności komponentów

Przepływ żądania HTTP (uproszczony)

PRZEGLĄDARKA/MOBILE WEB
↕ HTTPS
Next.js 14 (SSR + CSR) | Route Handler /api/v1/... (BFF proxy)
↕ HTTP REST ↕ WebSocket
Daphne ASGI
Django REST Framework | Django Channels (WebSocket Consumer)
↕ ORM/PostGIS ↕ Channel layer ↕ Cache/Queue
PostgreSQL+PostGIS | Redis 7 | Celery Worker + Beat
↕ HTTP API
Nominatim (geo) | Overpass (POI) | OpenAI (AI) | Google OAuth | SMTP

3.3 Przepływ typowego żądania REST

Każde żądanie od użytkownika przechodzi przez następujące etapy:

1. Użytkownik wykonuje akcję w przeglądarce (np. klika Szukaj)
2. Next.js kieruje żądanie do własnego Route Handlera (BFF) pod /api/v1/...
3. BFF przekazuje żądanie do Django (wewnętrzny URL INTERNAL_API_URL)
4. Daphne przyjmuje żądanie, middleware JWT weryfikuje token z nagłówka Authorization
5. Django sprawdza uprawnienia (IsAuthenticated, IsHost, IsAdmin) i throttling
6. Serwis wywołuje model/ORM, ewentualnie odpytuje Redis (cache) lub kolejkuje zadanie Celery
7. Odpowiedź JSON wraca przez Daphne → BFF → Next.js → przeglądarka

Dlaczego BFF (Backend-for-Frontend)?

Warstwa BFF w Next.js rozwiązuje kilka problemów jednocześnie: unika wystawiania wewnętrznego adresu Django bezpośrednio do przeglądarki, upraszcza obsługę błędów sieciowych (Django niedostępne → kod UPSTREAM_UNAVAILABLE), umożliwia spójne SSR bez problemów z CORS w

> środowisku Docker, oraz pozwala na dodatkową logikę transformacji danych przed wysłaniem do klienta.

3.4 Komunikacja w czasie rzeczywistym (WebSocket)

Czat między gościem a gospodarzem działa przez WebSockety obsługiwane przez Django Channels z dedykowanym consumerem asynchronicznym. Każda konwersacja ma własny kanał (group) w Redis.

Parametr	Wartość
Adres połączenia	ws[s]://host/ws/conversations/{uuid}/
Autoryzacja	Token JWT w query string: ?token=
Zdarzenia klienta → serwer	message.send, typing.start, typing.stop, message.read
Zdarzenia serwer → klient	message.new, typing.start, typing.stop, message.read
Warstwa kanałów	Redis 7 (channels-redis)
Serwer ASGI	Daphne — obsługuje jednocześnie HTTP i WebSocket

3.5 Zadania asynchroniczne (Celery)

Celery obsługuje zadania asynchroniczne (wywoływane po zdarzeniu) i cykliczne (harmonogram Beat). Broker i backend wyników to Redis.

Harmonogram	Zadanie
Co 12 minut	cleanup_expired_ai_sessions — sprzątanie wygasłych sesji AI
Co 22 minuty	cleanup_expired_compare_sessions — usuwanie wygasłych sesji porównywarki
Co 30 minut	Auto-anulowanie porzuconych rezerwacji AWAITING_PAYMENT → ABANDONED
Co godzinę	Auto-odrzucenie prośb hosta po przekroczeniu okna decyzyjnego → REJECTED
Codziennie 10:00	Przypomnienia e-mail o recenzji (7 dni po wymeldowaniu)
Codziennie 03:05	Odświeżanie cache POI z Overpass API (dane starsze niż 24h)
Codziennie 04:10	Odświeżanie podsumowań tekstowych obszarów (dane starsze niż 7 dni)
1. dnia miesiąca	Raport kosztów API OpenAI (transparentność operacyjna)
Na żądanie	Wysyłka e-maili, przeliczenie średniej oceny oferty, blind release recenzji

5. Stack technologiczny

4.1 Backend (backend/)

Obszar	Technologia	Opis / zastosowanie
Język	Python 3.12	Pełna zgodność z obrazem Docker; CI używa Python 3.12
Framework	Django 5.1.4	ORM, administracja, system uprawnień, signals, zarządzanie migracjami
REST API	Django REST Framework 3.x	Widoki API, serializery, paginacja cursorowa, throttling klas
Dokumentacja API	drf-spectacular	OpenAPI 3.0 + Swagger UI pod /api/schema/swagger-ui/
GIS	GeoDjango + PostGIS 3.4	Zapytania przestrzenne, PointField, indeksy GiST — wyszukiwanie geo
Autentykacja	SimpleJWT + Google OAuth	JWT: access token 60 min (HTTP-only cookie) + refresh 30 dni z rotacją; OAuth Google z auto-tworzeniem konta
Realtime	Django Channels + Daphne	WebSocket czatu, consumer asynchroniczny, channels-redis jako channel layer
Zadania async	Celery 5 + django-celery-beat	Worker + harmonogram Beat; broker Redis; zadania async i cykliczne
Cache	django-redis + Redis 7	Cache zapytań (5 min TTL dla wyszukiwania), sesje porównywarki, limity AI
Pliki / Storage	boto3 + django-storages	Opcjonalny storage S3 (USE_S3=True); lokalnie MEDIA_ROOT z WhiteNoise
Walidacja plików	Pillow + python-magic	Sprawdzanie typów MIME (nie tylko rozszerzenie), bezpieczne uploady obrazów
Monitoring	sentry-sdk[django]	Raportowanie błędów i wyjątków do Sentry (tylko produkcja)
Testy	pytest + pytest-django + Faker	25 plików testowych; testy jednostkowe i integracyjne z bazą PostGIS; Faker 30.8.1 + factory-boy
Linting	ruff	Szybki linter i formatter zgodny z PEP 8; skonfigurowany w ruff.toml

4.2 Frontend (frontend/)

Obszar	Technologia	Opis / zastosowanie
Framework	Next.js 14 (App Router)	SSR, ISR, RSC; React 18; TypeScript; 44 strony w 3 grupach tras
Stylowanie	Tailwind CSS	Utility-first CSS, projekt responsywny, Forest Premium design system
Komponenty UI	Radix UI	Dostępne, nieostylowane prymitywy (dialog, popover, select)

Mapy	Leaflet + react-leaflet + markercluster	Interaktywna mapa wyszukiwania, klasteryzacja markerów, GeoJSON overlay województw
Formularze	react-hook-form + zod	Walidacja po stronie klienta, typowana z TypeScript
Stan globalny	Zustand	Lekki store bez boilerplate Redux; osobne store dla auth, compare, search
JWT frontend	jose	Weryfikacja tokenów w Next.js middleware — ochrona tras chronionych
Animacje	framer-motion	Przejścia między stronami, animacje UI, efekty hover
Notyfikacje	react-hot-toast	Toast notifications dla akcji użytkownika
Testy E2E	Playwright	Testy end-to-end w Chromium/Firefox/WebKit; krytyczne ścieżki użytkownika

4.3 Infrastruktura i narzędzia

Narzędzie	Opis
Docker Compose	Lokalny stack: postgis:16-3.4, redis:7-alpine, backend (Daphne), frontend (Node 20); profil celery dla workerów
Makefile	Skróty: make dev, make migrate, make seed, make test, make lint, make superuser
GitHub Actions	CI: ruff + pytest z usługami PostGIS i Redis; Playwright E2E na gałęziach main/develop
Seed data	manage.py seed_db / seed_polska / seed_mass_listings — ładuje realistyczne dane demo z Faker

6. Moduły domenowe backendu

Backend jest podzielony na 12 aplikacji Django (apps/), każda odpowiadająca jednej domenie biznesowej. Wzorzec implementacji jest konsekwentny: widok → serializer → serwis → model. Logika biznesowa jest enkapsulowana w warstwach serwisowych, nie w widokach ani serializerach.

Moduł	Domena	Zakres odpowiedzialności
users	Użytkownicy	Model User (AbstractBaseUser, e-mail jako login), UserProfile (bio, avatar, preferencje językowe), WishlistItem, SavedSearch. Rejestracja email/hasło i Google OAuth. Soft delete, UUID PK.
common	Wspólne komponenty	BaseModel (UUID, soft delete, timestamps), AuditLog (ślad działań dla compliance), wyjątki API ze standardowymi kodami błędów, paginacja, middleware WebSocket JWT, EmailService, health check endpoint.
listings	Oferty noclegowe	Model Listing (status DRAFT→PENDING→APPROVED/REJECTED/ARCHIVED), ListingLocation (PointField + tagi geo: near_lake, near_mountains, near_sea, near_forest, beach_access, ski_slopes_nearby, quiet_rural, historic_center_nearby, cycling_routes_nearby), ListingImage, BlockedDate, destination_score_cache.
pricing	Silnik cenowy	CustomDatePrice (nadpisanie ceny na konkretny dzień), SeasonalPricingRule (zakres dat + mnożnik), HolidayPricingRule (święta), LongStayDiscountRule (rabat za długi pobyt). PricingService oblicza pełny breakdown dla podanych dat i gości.
bookings	Rezerwacje	Model Booking (statusy: PENDING, AWAITING_PAYMENT, CONFIRMED, COMPLETED, REJECTED, CANCELLED, ABANDONED), pricing_breakdown (JSON — snapshot ceny), BookingStatusHistory, Payment, StripeWebhookEvent.
search	Wyszukiwanie	SearchOrchestrator z geo-filtrowaniem PostGIS distance, filtrami tagowymi, TravelModeRanker (scoring per tryb podróży), 5-minutowy cache Redis. Geokodowanie przez Nominatim.
host	Panel gospodarza	Onboarding, CRUD ofert, upload zdjęć z throttlingiem, wysłanie do moderacji, zarządzanie rezerwacjami, reguły cenowe, powiadomienia, statystyki zarobków, recenzje po stronie hosta.
moderation	Moderacja treści	Kolejka ofert w statusie PENDING; akcje: approve (→ APPROVED) lub reject (→ REJECTED + komentarz). Dostępne tylko dla is_admin=True.
reviews	Recenzje	Mechanizm ślepego ujawnienia (blind release), subskory (czystość, lokalizacja, komunikacja, dokładność), odpowiedź hosta, przeliczanie average_rating oferty przez Celery.
messaging	Czat	Conversation (1:1 per listing-gość), Message z read_at, MessageTemplate (szablony z podstawianiem zmiennych: {{guest_name}}, {{listing_title}}), WebSocket consumer asynchroniczny.
ai_assistant	Asystent AI	AiTravelSession (TTL 24h), AiTravelPrompt, AiFilterInterpretation (sparsowane parametry),

		AiRecommendation (oferta + wyjaśnienie). Obsługa OpenAI i kompatybilnych API (np. Groq z Llama).
discovery	Odkrywanie ofert	DiscoveryCollection (kuratorowane kolekcje, sort_order, tryb podróży), CompareSession (maks. 3 oferty, TTL 48h, jedna sesja per zalogowanego użytkownika).
location_intelligence	Inteligencja lokalizacji	NearbyPlaceCache (wyniki Overpass, TTL 24h), AreaSummaryCache (opis obszaru tekstowy, TTL 7 dni). Scoring atrakcyjności okolicy (0–100 per tryb podróży) w Listing.destination_score_cache.

Model bazowy i soft delete

Każdy model dziedziczy po BaseModel z apps/common/. Klucz główny to UUID (nie auto-inkrementowany integer), co eliminuje ryzyko przewidywalnych ID w API i ułatwia przyszłą migrację. Pole deleted_at implementuje soft delete: rekord jest oznaczany jako usunięty zamiast fizycznego kasowania. Domyślny manager objects automatycznie filtruje rekordy z deleted_at IS NULL, a manager all_objects udostępnia pełny zbiór danych (dla administracji i audytu).

AuditLog — ślad działań

Dedykowany model w apps/common/ rejestruje kluczowe zdarzenia systemowe (rejestracja przez Google OAuth, logowanie, akcje moderacji) wraz z informacją o użytkowniku, typie akcji i znaczniku czasu. Ślad audytu jest niezbędny z perspektywy compliance i wsparcia technicznego.

7. Funkcjonalności — opis szczegółowy

6.1 Konta i uwierzytelnianie użytkowników

Platforma obsługuje dwa sposoby rejestracji i logowania: klasyczne konto e-mail/hasło oraz logowanie kontem Google (OAuth 2.0). Oba mechanizmy zwracają te same tokeny JWT, więc reszta systemu jest niezależna od metody autentykacji.

- Rejestracja e-mail — unikalny adres e-mail jest loginem; hasło hashowane przez Django (PBKDF2 z SHA256)
- Google OAuth 2.0 — token od Google weryfikowany po stronie backendu (google-auth); jeśli konto z tym e-mailem już istnieje, następuje automatyczne powiązanie
- JWT — access token (60 min) przechowywany w ciasteczku HTTP-only (ochrona przed XSS); refresh token (30 dni) z rotacją zapewnia przedłużenie sesji
- Role użytkownika — ten sam użytkownik może być jednocześnie gościem i gospodarzem (flagi is_host, is_admin na modelu User)
- Profil (UserProfile) — bio, avatar (upload z walidacją MIME), preferencje językowe, kraj
- Lista życzeń (WishlistItem) — gość zapisuje interesujące oferty; lista dostępna w koncie
- Zapisane wyszukiwania (SavedSearch) — przechowuje parametry wyszukiwania do ponownego użycia z opcją powiadomień
- AuthCrossTabSync — wylogowanie w jednej karcie przeglądarki wylogowuje wszystkie otwarte karty (synchronizacja między zakładkami)

6.2 Oferty i wyszukiwanie

Wyszukiwanie jest centralną funkcją produktu. Użytkownik może wpisać nazwę miejscowości, wybrać tryb podróży i zobaczyć oferty zarówno na mapie, jak i na liście. Klasteryzacja markerów zapewnia czytelność przy dużej liczbie wyników.

Cykl życia oferty

Status	Znaczenie
DRAFT	Szkic tworzony przez hosta; niewidoczny publicznie; może być dowolnie edytowany
PENDING	Po kliknięciu 'Wyślij do moderacji'; oczekuje na zatwierdzenie administratora; zablokowany do edycji
APPROVED	Zaakceptowana przez moderatora; widoczna w wynikach wyszukiwania; goście mogą rezerwować
REJECTED	Odrzucona z komentarzem moderatora; host może edytować i ponownie wysłać
ARCHIVED	Wycofana z oferty przez hosta; zachowana w historii, niewidoczna publicznie

Tryby podróży i scoring

Każda oferta jest scorowana względem 9 trybów podróży. TravelModeRanker przypisuje punkty na podstawie udogodnień (amenities) i tagów geograficznych lokalizacji.

Tryb	Profil i charakterystyka
romantic	Pary — cisza, prywatność, kominek, jacuzzi, wino; scoring wyższy za izolację i luksusowe udogodnienia

family	Rodziny z dziećmi — bezpieczna przestrzeń, plac zabaw, kuchnia; POI: atrakcje rodzinne
pet	Podróże ze zwierzętami — duży ogród, tereny zielone w okolicy; flaga pet_friendly
workation	Praca zdalna — szybkie Wi-Fi, biurko, zaciszne otoczenie; cisza i przestrzeń do pracy
slow	Slow travel — cisza, natura, detox od miejskiego zgiełku; priorytet: POI natura i rezerwaty
outdoor	Aktywny wypoczynek — szlaki, rowery, kajaki; POI: szlaki turystyczne, wypożyczalnie
lake	Mazury i jeziora — dostęp do wody, łódki, rejsy; tag near_lake, beach_access
mountains	Góry — widoki, szlaki górskie, kolej linowa; tagi near_mountains, ski_slopes_nearby
wellness	Wellness — sauna, basen, spa, masaże; scoring wyższy za udogodnienia zdrowotne

6.3 Silnik cen i rezerwacje

Cennik w StayMap Polska jest jednym z najbardziej zaawansowanych elementów systemu. Uwzględnia sezonowość, polskie święta, niestandardowe szczyty turystyczne, liczbę gości oraz długość pobytu.

Wzór kalkulacji ceny pobytu

Krok	Składnik	Obliczenie
1	Cena za noc	cena_bazowa × mnożnik_sezonowy × mnożnik_świąteczny
2	Suma noclegów	Σ(cena_za_noc) dla każdej nocy pobytu
3	Dopłaty za gości	Σ(cena_noc × liczba_dodatkowych_dorosłych × 10%) + Σ(cena_noc × liczba_dzieci × 5%)
4	Suma pośrednia	Suma noclegów + dopłaty za gości
5	Rabat za długi pobyt	suma_pośrednia × procent_rabatu (jeśli liczba_nocy ≥ próg LongStayDiscountRule)
6	Opłata za sprzątanie	Stała kwota ustalona przez hosta (cleaning_fee)
7	Prowizja platformy	(suma_po_rabacie + sprzątanie) × % prowizji (domyślnie 15%)
SUMA	**RAZEM DO ZAPŁATY**	**suma_po_rabacie + sprzątanie + prowizja_platformy**

Obsługa polskich świąt i sezonowości

Typ reguły	Opis i konfiguracja
Ustawowe dni wolne (GUS)	Automatyczne — zaszyte w kodzie HolidayPricingService; bez konfiguracji; obejmują Nowy Rok, Wielkanoc, Boże Ciało, Wniebowzięcie, Wszystkich Świętych, Święto Niepodległości, Boże Narodzenie i inne

Dodatkowe szczyty turystyczne	Opcjonalne, aktywowane przez admina: Dzień Kobiet (8 mar), Andrzejki (30 lis), Wszystkich Świętych (1 lis); flaga apply_pl_travel_peak_extras
Reguły sezonowe hosta	SeasonalPricingRule — zakres dat + mnożnik (np. lato 1.12×, zima 1.10×); dowolna liczba reguł per oferta
Niestandardowe daty hosta	HolidayPricingRule — host definiuje własne specjalne daty z indywidualnym mnożnikiem (np. jarmark świąteczny)
Nadpisanie konkretnego dnia	CustomDatePrice — stała cena na wybrany dzień, ignoruje wszystkie mnożniki (priorytet najwyższy)

Tryby rezerwacji

Tryb	Opis przepływu
INSTANT (natychmiastowa)	Rezerwacja tworzona ze statusem PENDING → AWAITING_PAYMENT; po opłaceniu → CONFIRMED; automatyczne anulowanie po BOOKING_PAYMENT_TIMEOUT_H (domyślnie 1h)
REQUEST (na prośbę)	Rezerwacja tworzona ze statusem PENDING; host musi zaakceptować w oknie HOST_REQUEST_ACCEPT_HOURS (domyślnie 24h); brak odpowiedzi → auto-odrzucenie przez Celery Beat

Snapshot cenowy (pricing_breakdown)

W momencie tworzenia rezerwacji pełny breakdown cenowy jest zapisywany jako JSON (pole pricing_breakdown na modelu Booking). Zmiana cennika przez hosta po dokonaniu rezerwacji nie wpływa na już zatwierdzone pobyty. Jest to kluczowe zarówno z perspektywy prawnej (umowa z gościem) jak i rozliczeń finansowych.

6.4 Panel gospodarza i onboarding

Gospodarz ma do dyspozycji dedykowany panel z pełnym zestawem narzędzi do zarządzania działalnością. Wejście do panelu wymaga przejścia jednorazowego onboardingu — aktywacji profilu hosta.

- Onboarding — jednorazowe uruchomienie profilu hosta (POST /host/onboarding/start/); od tej chwili użytkownik ma rolę hosta i dostęp do panelu
- Zarządzanie ofertami — tworzenie, edycja, archiwizacja; upload zdjęć z limitem i throttlingiem; walidacja MIME przez Pillow i python-magic
- Wysłanie do moderacji — jednym kliknięciem zmiana statusu DRAFT → PENDING
- Reguły cenowe — tworzenie/edycja reguł sezonowych, świątecznych i long-stay discount przez API panelu hosta
- Kalendarz dostępności — blokowanie dat (BlockedDate), podgląd rezerwacji na kalendarzu
- Zarządzanie rezerwacjami — lista, podgląd szczegółów, akceptacja/odrzucenie (tryb REQUEST)
- Szablony wiadomości (MessageTemplate) — predefiniowane odpowiedzi z dynamicznymi zmiennymi {{guest_name}}, {{listing_title}} przyspieszające komunikację
- Recenzje — podgląd ocen ofert i możliwość jednorazowej odpowiedzi hosta na każdą recenzję
- Statystyki zarobków — widok earnings/payouts (endpoint /host/earnings/)

6.5 Moderacja treści

Zanim oferta trafi do wyników wyszukiwania, musi przejść przez ręczną moderację. Mechanizm zapewnia, że goście widzą tylko zweryfikowane, kompletne oferty.

- Administrator widzi kolejkę ofert w statusie PENDING z pełnymi szczegółami

- Może zatwierdzić ofertę (→ APPROVED, natychmiastowa widoczność) lub odrzucić z komentarzem (→ REJECTED)
- Host otrzymuje informację o odrzuceniu wraz z uzasadnieniem i może edytować i ponownie wysłać
- Endpointy moderacji są zabezpieczone uprawnieniem is_admin=True — gość ani host nie mają do nich dostępu

6.6 System recenzji — mechanizm ślepego ujawnienia

System recenzji zapobiega sytuacji, w której jedna ze stron odczyta ocenę drugiej i dopiero wtedy napisze swoją własną. Mechanizm blind release eliminuje ten problem:

8. Gość ocenia pobyt (recenzja GUEST, is_public=False) — niewidoczna dla hosta
9. Host ocenia gościa (recenzja HOST, is_public=False) — niezależnie od recenzji gościa
10. Gdy obydwie strony złożyły recenzje, Celery task release_blind_review_task ujawnia je jednocześnie (is_public=True) w atomowej transakcji bazodanowej
11. Jeśli jedna ze stron nie oceni w oknie czasowym (blind_release_at), jej recenzja pozostaje ukryta; recenzja istniejącej strony jest ujawniana po terminie

Recenzja gościa zawiera ogólną ocenę (1–5 gwiazdek) i opcjonalne subskory: czystość, lokalizacja, komunikacja z gospodarzem, zgodność z opisem. Host może raz odpowiedzieć na recenzję gościa — odpowiedź jest widoczna publicznie. Przypomnienia e-mail o recenzji są wysyłane 7 dni po wymeldowaniu przez Celery.

6.7 Komunikacja i czat WebSocket

Każda para gość–oferta może mieć jedną konwersację. Wiadomości są dostępne zarówno przez REST API (historia, tworzenie) jak i przez WebSocket (powiadomienia na żywo).

- Konwersacja tworzona automatycznie przy pierwszym kontakcie gościa z daną ofertą
- REST API: lista konwersacji z licznikiem nieprzeczytanych, historia wiadomości z paginacją
- WebSocket: nowe wiadomości pojawiają się natychmiastowo bez odświeżania strony
- Wskaźniki pisania (typing indicators) — event typing.start/stop widoczny dla rozmówcy w czasie rzeczywistym
- Potwierdzenia przeczytania (read receipts) — event message.read aktualizuje pole read_at
- Szablony wiadomości dla hosta — predefiniowane odpowiedzi z podstawionymi danymi gościa i oferty
- Licznik nieprzeczytanych wiadomości widoczny w nawigacji aplikacji dla zalogowanego użytkownika

6.8 Asystent AI — wyszukiwanie w języku naturalnym

Moduł AI pozwala użytkownikowi wpisać zapytanie po polsku, tak jak zadałby pytanie człowiekowi — bez wypełniania formularzy z filtrami.

Etap	Opis procesu
Zapytanie użytkownika	Przykład: 'Szukam spokojnego domku dla 4 osób na Mazurach na tydzień w sierpniu, z sauną, do 300 zł/noc'
Parsowanie przez LLM	Model językowy (OpenAI gpt-4o-mini lub kompatybilny np. Groq/Llama) wyodrębnia: lokalizacja=Mazury, tryb=lake, goście=4, daty=sierpień, amenity=sauna, max_price=300
Wyszukiwanie	SearchOrchestrator uruchamia geo-filtrowanie + scoring trybów podróży

	(zawsze świeże dane, bez cache)
Wyjaśnienie dopasowania	Drugi call do LLM generuje 2-3 zdaniowe wyjaśnienie po polsku dla każdej rekomendacji + lista wyróżników
Wynik	Frontend wyświetla oferty z kartami wyjaśnienia: 'Ten domek spełnia Twoje oczekiwania, ponieważ...'

Sesje AI mają TTL 24h. Użytkownik może zadawać pytania uzupełniające (follow-up prompts). System obsługuje OpenAI i alternatywy kompatybilne z API (Groq z Llama — tańsza opcja). Zmiana dostawcy wymaga ustawienia OPENAI_BASE_URL i OPENAI_MODEL.

6.9 Odkrywanie ofert i porównywarka

Moduł discovery obsługuje dwa scenariusze: kuratorowane kolekcje tematyczne na stronie głównej oraz sesję porównawczą ofert.

- Kolekcje kuratorowane (DiscoveryCollection) — administrator tworzy tematyczne zestawy ofert (np. 'Górskie chaty z sauną', 'Last minute Bałtyk') z określoną kolejnością i filtrem trybu podróży
- Oferty last-minute — dynamiczny feed propozycji z krótkim terminem dostępności
- Porównywarka — gość może zebrać do 3 ofert w sesji porównawczej; zestawienie side-by-side: cena, lokalizacja, amenities, oceny
- Sesja działa dla anonimowych (klucz w localStorage) i zalogowanych (1 aktywna sesja per użytkownik w bazie)
- TTL sesji: 48h; wygasłe sesje czyszczone przez Celery co 22 minuty; max 3 oferty egzekwowane po stronie API i frontu

6.10 Inteligencja lokalizacji (POI & scoring)

Każda oferta może być wzbogacona o automatycznie generowane informacje o okolicy, bez manualnego opisywania przez hosta. Dane pochodzą z OpenStreetMap (Overpass API).

Kategoria POI	Przykłady punktów zainteresowania
eat_drink	Restauracje, kawiarnie, bary, puby, lodziarnie
nature_leisure	Parki, rezerwaty przyrody, tereny rekreacyjne, lasy
family	Atrakcje dla dzieci, place zabaw, aquaparki, zoo
culture	Muzea, zabytki, galerie, kościoły, zamki
transport	Stacje kolejowe, przystanki autobusowe, lotniska
outdoor	Szlaki turystyczne, wypożyczalnie rowerów, kajakarnie, mariny
services	Sklepy spożywcze, apteki, bankomaty, stacje paliw

Wyniki Overpass są cache'owane przez 24h (NearbyPlaceCache) i odświeżane nocą przez Celery. Na podstawie gęstości POI obliczany jest Destination Score (0–100 punktów per tryb podróży), który wpływa na ranking ofert w wyszukiwarce. Tekstowe podsumowanie okolicy (AreaSummaryCache, TTL 7 dni) jest wyświetlane na karcie oferty.

8. Frontend — mapa aplikacji i widoki

Frontend zbudowany jest w Next.js 14 z App Router. Każda trasa może być renderowana po stronie serwera (SSR), po stronie klienta (CSR) lub jako statyczna strona (SSG). Aplikacja zawiera 44 strony podzielone na trzy grupy tras.

7.1 Grupy tras

Grupa	Przykładowe URL	Charakter i zabezpieczenia
(auth)	/login, /register	Logowanie i rejestracja; bez paska nawigacji hosta; niezalogowani only
(main)	/, /search, /listing/[slug], /bookings, /account, /messages, /compare, /ai, /discovery, /wishlist, /noclegi/[region], /travel/[mode]	Główna witryna dla gościa; SSR dla SEO na stronach ofert; niektóre trasy wymagają IsAuthenticated
(host)	/host/dashboard, /host/listings, /host/new-listing, /host/bookings, /host/messages, /host/reviews, /host/pricing, /host/earnings, /host/settings, /host/calendar, /host/payouts	Panel gospodarza; wymaga is_host=True; dostęp chroniony middleware Next.js (jose JWT)

7.2 Strony i widoki — opis szczegółowy

Strona główna (/)

Centralna strona odkrywania ofert z bogatą strukturą sekcji:

- Hero z wyszukiwarką — search bar z wyszukiwaniem lokalizacji, datami, liczbą gości; efekt glow ring i shimmer; ikony trybów podróży z badges
- Kafle regionów — Góry, Bałtyk, Jeziora, Uzdrowiska z linkami do /noclegi/[region]
- Kafle trybów podróży — 9 trybów z ikonami prowadzące do /travel/[mode]
- Teaser AI — sekcja promująca asystenta naturalnego języka
- Kuratorowane kolekcje — DiscoveryCollection z sortowaniem i tematami
- Last-minute — 3D carousel ofert z krótkim terminem dostępności i licznikiem czasu

Wyszukiwanie (/search)

- Mapa Leaflet jako główny element — klasteryzacja markerów przy dużej liczbie wyników
- Panel filtrów boczny — lokalizacja, daty, goście, tryb podróży, cena (min/max), udogodnienia
- Paginacja cursorowa — ładowanie większej liczby wyników przy scrollu (infinite scroll)
- GeoJSON overlay — granice województw i główne miasta Polski jako warstwy na mapie
- Widok listy równoległy z mapą — użytkownik może przełączać widoki

Karta oferty (/listing/[slug])

- Galeria zdjęć z pełnoekranowym lightboxem
- Opis, udogodnienia z ikonami, polityka anulowania, zasady pobytu
- Widget rezerwacyjny — live pricing (POST /bookings/quote/) po zmianie dat; przycisk Zarezerwuj
- Recenzje z podskores (czystość, lokalizacja, komunikacja, dokładność) i odpowiedziami hosta
- Opis okolicy + interaktywna mapa POI (punkty zainteresowania w okolicy)
- Kalendarz cen i dostępności

- Przycisk Dodaj do porównywarki

Panel gospodarza (/host/*)

Strona	Zawartość i funkcje
/host/dashboard	Główny panel: quick-stats (liczba ofert, aktywne rezerwacje, nowe wiadomości), powiadomienia, szybkie linki do akcji
/host/listings	Lista ofert z statusami; CRUD; wysyłanie do moderacji; podgląd statusu każdej oferty
/host/new-listing	Kreator nowej oferty: tytuł, opis, lokalizacja, upload zdjęć (z throttlingiem), udogodnienia, typ noclegu, tagi geo
/host/bookings	Lista rezerwacji z filtrem statusu; akceptuj/odrzuć dla trybu REQUEST; podgląd szczegółów i pricing_breakdown
/host/bookings/pending	Filtrowany widok rezerwacji oczekujących na decyzję hosta (tryb REQUEST)
/host/bookings/confirmed	Widok potwierdzonych rezerwacji z datami pobytów
/host/calendar	Kalendarz dostępności — blokowanie dat, podgląd rezerwacji w układzie miesięcznym
/host/pricing	Tworzenie i edycja reguł sezonowych, świątecznych i long-stay discount; podgląd kalendarza cen
/host/messages	Lista konwersacji z gośćmi; WebSocket dla wiadomości na żywo; szablony wiadomości
/host/reviews	Podgląd recenzji ofert; formularz jednorazowej odpowiedzi hosta na recenzję gościa
/host/earnings	Statystyki zarobków, historia transakcji, podsumowanie prowizji platformy
/host/payouts	Historia wypłat (po konfiguracji Stripe Connect lub tradycyjnego przelewu)
/host/notifications	Centrum powiadomień: nowe rezerwacje, wiadomości, akceptacje/odrzucenia moderacji
/host/settings	Ustawienia profilu hosta, preferencje powiadomień, dane kontaktowe
/host/profile	Publiczny profil gospodarza — bio, avatar, opinie od gości
/host/listing-submitted	Strona potwierdzenia po wysłaniu oferty do moderacji

Pozostałe strony główne

Strona	Zawartość i funkcje
/account	Dane osobowe, avatar, preferencje językowe, historia rezerwacji, lista życzeń, zapisane wyszukiwania
/bookings	Historia wszystkich rezerwacji gościa z filtrami statusu i możliwością anulowania
/bookings/[id]/success	Strona potwierdzenia po pomyślnej rezerwacji z danymi pobytu
/messages	Lista konwersacji gościa (licznik nieprzeczytanych); WebSocket; typing indicators; read receipts
/compare	Tabela side-by-side do 3 ofert: cena, lokalizacja, amenities, oceny;

	persystentna sesja
/ai	Pole tekstowe naturalnego języka; przetwarzanie zapytania; wyniki z wyjaśnieniami dopasowania
/ai/history	Historia sesji AI z poprzednimi zapytaniami i wynikami (AiTravelSession)
/discovery	Kolekcje tematyczne; oferty last-minute; filtry trybu podróży; karuzela ofert
/wishlist	Lista zapisanych ofert użytkownika z możliwością usunięcia i dodania do porównywarki
/noclegi/[region]	SSG — strony SEO per region (np. /noclegi/mazury, /noclegi/zakopane) z ofertami i opisem regionu
/travel/[mode]	SSG — strony SEO per tryb podróży (np. /travel/family, /travel/wellness) z dopasowanymi ofertami
/host/onboarding	Kreator aktywacji profilu hosta — jednorazowy, wymagany przed dostępem do panelu
/login, /register	Formularze autentykacji z opcją logowania Google OAuth; walidacja Zod; react-hook-form
/kariera, /kontakt, /pomoc, /polityka-prywatnosci, /regulamin	Strony informacyjne — kariera, kontakt, centrum pomocy, polityka prywatności, regulamin platformy

7.3 BFF — Backend for Frontend

Plik frontend/src/app/api/v1/[...path]/route.ts pełni rolę cienkiej warstwy proxy między przeglądarką a Django. Wszystkie żądania REST z frontu przechodzą przez ten handler, który przekazuje nagłówek Authorization (JWT), obsługuje GET/POST/PUT/PATCH/DELETE/HEAD, przesyła JSON i multipart/form-data (upload plików), zwraca HTTP 503 z kodem UPSTREAM_UNAVAILABLE jeśli Django jest nieosiągalne, oraz zachowuje nagłówki Rate-Limit i Retry-After z odpowiedzi Django.

9. API — przegląd endpointów

Pełna dokumentacja API jest dostępna jako Swagger UI pod /api/schema/swagger-ui/ (generowana automatycznie przez drf-spectacular). Poniżej zestawienie kluczowych grup endpointów.

Grupa	Kluczowe endpointy	Co realizuje
Autentykacja	POST /auth/register/ | POST /auth/token/ | POST /auth/token/refresh/ | POST /auth/google/ | GET/PATCH /auth/me/	Rejestracja, logowanie (e-mail+hasło i Google OAuth), odśwież token JWT, profil zalogowanego użytkownika
Oferty (publiczne)	GET /listings/search/ | GET /listings/{slug}/ | GET /listings/{slug}/price-calendar/	Wyszukiwanie z geo-filtrowaniem, szczegóły oferty (slug dla SEO), kalendarz cen per oferta
Rezerwacje (gość)	POST /bookings/quote/ | POST /bookings/ | GET /bookings/me/ | GET /bookings/{uuid}/ | DELETE /bookings/{uuid}/	Wycena (bez tworzenia), nowa rezerwacja, lista własnych rezerwacji, szczegóły, anulowanie
Recenzje	POST /reviews/ | PATCH /reviews/{uuid}/host-response/	Wystawienie recenzji (gość lub host), odpowiedź hosta na recenzję gościa
Wiadomości	GET/POST /conversations/ | GET/POST /conversations/{uuid}/messages/ | WS /ws/conversations/{uuid}/	Lista/tworzenie konwersacji, historia/wysyłka wiadomości, WebSocket dla czatu na żywo
Panel hosta	POST /host/onboarding/start/ | GET/POST /host/listings/ | POST /host/listings/{uuid}/images/ | POST .../submit-for-review/ | GET /host/bookings/ | PATCH /host/bookings/{uuid}/status/	Pełny CRUD ofert, upload zdjęć, moderacja, zarządzanie rezerwacjami, reguły cenowe, statystyki zarobków
Moderacja (admin)	GET /admin/moderation/listings/ | POST .../approve/ | POST .../reject/	Kolejka PENDING, zatwierdzenie, odrzucenie z komentarzem — wyłącznie is_admin=True
Discovery & Compare	GET /discovery/ | GET/POST /compare/ | POST /compare/bootstrap/ | POST /compare/listings/ | DELETE /compare/listings/{uuid}/	Feed strony głównej, sesja porównawcza, dodawanie/usuwanie ofert do porównania
AI & Search	GET /search/ | POST /ai/search/ | GET /ai/search/{session_id}/ | POST /ai/search/{session_id}/prompt/	Zaawansowane wyszukiwanie geo, nowa sesja AI, podgląd sesji, pytanie uzupełniające
Konto użytkownika	POST/DELETE /wishlist/ | GET /saved-searches/ | POST /saved-searches/	Lista życzeń, zarządzanie zapisanymi wyszukiwaniami
System	GET /health/ | GET /api/schema/ | GET /api/schema/swagger-ui/	Health check, schemat OpenAPI, Swagger UI z pełną dokumentacją interaktywną

Standardy API

- Wszystkie odpowiedzi używają ujednoliconego formatu JSON z polami: data, errors, pagination

- Błędy mają standaryzowane kody: UPSTREAM_UNAVAILABLE, BOOKING_CONFLICT, LISTING_NOT_APPROVED, PAYMENT_FAILED i inne
- Throttling skonfigurowany na wrażliwych endpointach (upload zdjęć, operacje AI, autoryzacja) przez DRF throttle classes
- Paginacja cursorowa (wydajniejsza od offset dla dużych zbiorów danych przy wyszukiwaniu geograficznym)
- Wszystkie ID w API to UUID — eliminuje przewidywalne identyfikatory (ochrona przed IDOR)

10. Bezpieczeństwo systemu

Obszar	Mechanizm	Wartość bezpieczeństwa
Autentykacja	JWT: access 60 min w cookie HTTP-only; refresh 30 dni z rotacją; brak sesji serwerowej	Tokeny niedostępne przez JavaScript (XSS mitigation); rotacja minimalizuje ryzyko przejęcia refresh tokenu
Autoryzacja	Uprawnienia per endpoint: IsAuthenticated, IsHost, IsAdmin, IsAuthenticatedOrReadOnly; izolacja danych	Gość nie edytuje cudzych ofert; host nie widzi rezerwacji innych hostów; admin-only endpoints
Soft delete	Rekordy oznaczane deleted_at zamiast fizycznego kasowania	Ochrona przed przypadkową utratą danych; możliwość przywrócenia przez admina
Audit Log	AuditLog rejestruje kluczowe zdarzenia z user_id, action_type, timestamp	Ślad audytu dla compliance; diagnostyka incydentów bezpieczeństwa
Walidacja plików	Pillow weryfikuje zawartość graficzną; python-magic sprawdza rzeczywisty typ MIME	Ochrona przed szkodliwymi plikami ukrytymi pod rozszerzeniem *.jpg
Throttling	Limity na endpointach autoryzacji, uploadu zdjęć i AI	Ochrona przed brute-force i kosztownymi nadużyciami API OpenAI
Sentry (prod)	Automatyczne przechwytywanie wyjątków z pełnymi stack trace	Szybka diagnostyka problemów produkcyjnych bez dostępu do logów serwera
AuthCrossTabSync	Wylogowanie we wszystkich kartach przeglądarki jednocześnie	Zapobiega pozostawieniu aktywnej sesji na współdzielonym komputerze
UUID jako PK	Wszystkie ID w API to UUID zamiast auto-inkrementowanych integerów	Eliminuje przewidywalne ID (IDOR mitigation); utrudnia wyliczanie zasobów
Hasła	Django PBKDF2 SHA256 z solą; walidatory: podobieństwo, długość, popularność, numeryczność	Bezpieczne hashowanie zgodne ze standardami Django auth

Uwaga konfiguracyjna — krytyczna

JWT_SECRET po stronie Next.js musi być identyczny z Django SECRET_KEY. Niespójność tych wartości powoduje, że middleware Next.js nie może zweryfikować tokenu z ciasteczka — efektem są błędy 500 przy SSR. Należy to bezwzględnie sprawdzić przed wdrożeniem produkcyjnym.

11. Operacje i wdrożenie produkcyjne

10.1 Komponenty wymagane na produkcji

Komponent	Uwagi produkcyjne
PostgreSQL 16 + PostGIS	Źródło prawdy; wymaga rozszerzenia postgis i btree_gist; zalecane managed DB z automatycznym backupem i replikami read-only
Redis 7	Cache zapytań, broker Celery, warstwa kanałów WebSocket; jedna instancja obsługuje wszystkie trzy role; Redis Sentinel/Cluster dla HA
Celery Worker	Przynajmniej jeden worker obsługujący kolejkę domyślną; możliwość skalowania horyzontalnego
Celery Beat	Jeden scheduler (singleton!); uruchamianie więcej niż jednego Beat spowodowałoby duplikaty zadań
Daphne/Gunicorn	Dev: Daphne obsługuje HTTP+WS. Prod: Gunicorn dla HTTP i osobny Daphne/Uvicorn dla WS, lub unified ASGI
Nginx (reverse proxy)	TLS termination, serwowanie plików statycznych (WhiteNoise jako fallback), proxying do Gunicorn i Daphne
SMTP	Transakcyjne e-maile: potwierdzenia rezerwacji, przypomnienia recenzji; Gmail App Password lub SendGrid
S3 (opcjonalnie)	Skalowalny storage zdjęć (USE_S3=True); przy False pliki lądują w MEDIA_ROOT z WhiteNoise
Sentry	DSN przez SENTRY_DSN i SENTRY_ENVIRONMENT; tylko w produkcji (requirements/production.txt)

10.2 Kluczowe zmienne środowiskowe

Zmienna	Opis
SECRET_KEY	Silny losowy klucz Django; używany również jako JWT_SECRET po stronie Next.js (musi być identyczny!)
DATABASE_URL	postgres://user:pass@host:5432/dbname — musi być z rozszerzeniem PostGIS
REDIS_URL	redis://:pass@host:6379/0
OPENAI_API_KEY / OPENAI_BASE_URL / OPENAI_MODEL	Klucz OpenAI lub kompatybilnego dostawcy (Groq); puste BASE_URL = OpenAI; domyślny model: gpt-4o-mini
GOOGLE_OAUTH_CLIENT_ID	Z Google Cloud Console; wymagany dla logowania Google OAuth
USE_S3 / AWS_*	USE_S3=True aktywuje S3; wymagane: AWS_ACCESS_KEY_ID, AWS_SECRET_ACCESS_KEY, AWS_STORAGE_BUCKET_NAME
EMAIL_HOST / EMAIL_HOST_PASSWORD	SMTP host i hasło (dla Gmail: 16-znakowe App Password z 2FA aktywnym)
PLATFORM_SERVICE_FEE_PERCENT	Prowizja platformy (domyślnie 15); zmiana wymaga przemyślenia modelu biznesowego
HOST_REQUEST_ACCEPT_	Okno decyzyjne hosta w trybie REQUEST (domyślnie 24h)

HOURS	
AI_SESSION_TTL_HOURS / COMPARE_SESSION_TTL_HOURS	Czas życia sesji AI (24h) i porównywarki (48h) — konfigurowalne

10.3 Proces wdrożenia (kroki)

12. Sklonuj repozytorium, skopiuj .env.example do .env i uzupełnij wartości produkcyjne
13. Zbuduj obrazy Docker: docker compose -f docker-compose.prod.yml build
14. Wykonaj migracje bazy: docker compose run --rm backend python manage.py migrate
15. Zbierz pliki statyczne: python manage.py collectstatic --noinput
16. Uruchom stos: docker compose up -d (z profilem celery dla workerów)
17. Uruchom Next.js: npm run build && npm start (lub przez PM2/Docker)
18. Skonfiguruj Nginx jako reverse proxy z TLS (Let's Encrypt / certbot)
19. Zweryfikuj health check: GET /health/ powinien zwrócić HTTP 200
20. Sprawdź Sentry — po pierwszym uruchomieniu wyślij testowy wyjątek
21. Stwórz konto admina: python manage.py superuser lub przez panel Django Admin

12. Testy i CI/CD

11.1 Backend — testy jednostkowe i integracyjne

Backend testowany jest za pomocą pytest z pytest-django. 25 plików testowych obejmuje wszystkie kluczowe serwisy i endpointy. Testy korzystają z rzeczywistej bazy PostGIS (uruchamianej w Docker na CI).

- Testy jednostkowe serwisów — PricingService (kalkulacja cen, święta, rabaty), BookingService (konflikty, anulowania), AISearchService (parsowanie zapytań)
- Testy integracyjne endpointów — TestClient Django REST Framework dla wszystkich grup API
- Testy modeli — właściwości, relacje, soft delete, UUID jako PK
- Testy zadań Celery — weryfikacja logiki bez rzeczywistego kolejkowania
- Biblioteki testowe: pytest 8.3.3, pytest-django 4.9.0, pytest-cov 6.0.0, factory-boy 3.3.1, Faker 30.8.1

11.2 Frontend — testy E2E (Playwright)

Playwright uruchamia testy w Chromium/Firefox/WebKit, testując krytyczne ścieżki użytkownika end-to-end:

- Logowanie i rejestracja (e-mail + Google OAuth)
- Wyszukiwanie ofert z filtrami — wybór lokalizacji, dat, trybu podróży
- Przepływ rezerwacji — od wyboru dat do potwierdzenia (quote → booking → success)
- Tworzenie oferty przez hosta — kreator, upload zdjęć, wysłanie do moderacji
- Wysyłanie wiadomości w czacie przez WebSocket

11.3 Pipeline CI/CD (GitHub Actions)

Aspekt CI	Szczegóły konfiguracji
Wyzwalacz	Push lub pull request do gałęzi main / develop
Usługi kontenerowe	postgis:16-3.4 + redis:7 uruchamiane automatycznie przez GitHub Actions services
Job Backend	1. Lint: ruff check apps/ config/ | 2. Pytest: pytest -q --cov z bazą PostGIS, Python 3.12, GDAL, libmagic
Job Frontend	npm ci + npx playwright install + playwright test w Chromium/Firefox
Brama jakości	Każdy pull request musi przejść linting i testy; merge zablokowany przy błędach
Czas pipeline	Ok. 3–5 minut dla typowego PR

13. Zależności zewnętrzne

Usługa	Rola w systemie	Uwagi operacyjne
OpenStreetMap / Nominatim	Geokodowanie: adres lub nazwa miejsca → współrzędne GPS przy wyszukiwaniu	Bezpłatny; limit 1 req/s; wymagany poprawny User-Agent; nie do masowego geokodowania
Overpass API (OSM)	Pobieranie POI w promieniu od oferty; wyniki cache'owane 24h w NearbyPlaceCache	Bezpłatny; latencja 1–5s; cache jest kluczowy dla responsywności; nocne odświeżanie
OpenAI API (lub kompatybilny)	Parsowanie zapytań naturalnych; generowanie wyjaśnień dopasowania; domyślnie gpt-4o-mini	Koszt per token; monitoring w AiTravelSession.tokens_used; można zastąpić Groq (Llama) dla niższych kosztów
Google OAuth 2.0	Logowanie kontem Google; token weryfikowany po stronie backendu przez google-auth	Wymaga GOOGLE_OAUTH_CLIENT_ID z Google Cloud Console; konfiguracja OAuth consent screen
Stripe (przygotowany)	Płatności online; infrastruktura modeli (Payment, StripeWebhookEvent) gotowa	Wymaga konfiguracji konta Stripe, STRIPE_SECRET_KEY i webhook URL; decyzja biznesowa o modelu prowizji
SMTP (e-mail)	Transakcyjne e-maile: potwierdzenia rezerwacji, przypomnienia o recenzjach	Zalecane: Gmail App Password lub SendGrid; konfiguracja w EMAIL_HOST/EMAIL_HOST_PASSWORD

14. Diagramy przepływów

13.1 Przepływ rezerwacji — tryb INSTANT

13.1 Przepływ rezerwacji — tryb INSTANT	
Gość	Wybiera daty, liczbę gości, klika Zarezerwuj na karcie oferty
Frontend	POST /bookings/quote/ → wyświetla rozbicie ceny (PricingService)
Gość	Potwierdza rezerwację (przycisk Potwierdź)
Frontend	POST /bookings/ → tworzy rezerwację z pricing_snapshot
Backend	AvailabilityService sprawdza konflikty; PricingService zapisuje snapshot ceny
Backend	Rezerwacja: status PENDING → AWAITING_PAYMENT
Celery	Wysyłka e-mail potwierdzenia do gościa i hosta
Gość	Opłaca rezerwację (Stripe lub inny provider)
Backend	Po płatności: AWAITING_PAYMENT → CONFIRMED
Celery	Auto-anulowanie jeśli płatność nie nastąpi w BOOKING_PAYMENT_TIMEOUT_H (domyślnie 1h)
Celery	7 dni po wymeldowaniu (COMPLETED): przypomnienie e-mail o recenzji

13.2 Przepływ rezerwacji — tryb REQUEST

13.2 Przepływ rezerwacji — tryb REQUEST	
Gość	Tworzy prośbę o rezerwację (POST /bookings/)
Backend	Status: PENDING; ustawia host_response_deadline = teraz + HOST_REQUEST_ACCEPT_HOURS
Host	Widzi prośbę w panelu /host/bookings/pending; ma okno czasowe na decyzję
Host	Akceptuje → Status: PENDING → AWAITING_PAYMENT; gość opłaca
Host	Odrzuca → Status: PENDING → REJECTED; Celery powiadamia gościa
Celery Beat	Co godzinę: auto-odrzucenie po przekroczeniu deadline → REJECTED
Backend	Po akceptacji i płatności: identycznie jak tryb INSTANT od kroku płatności

13.3 Przepływ moderacji oferty

13.3 Przepływ moderacji oferty	
Host	Tworzy ofertę (DRAFT); uzupełnia informacje, dodaje zdjęcia
Host	Klika 'Wyślij do moderacji' → status: DRAFT → PENDING
Admin	Widzi ofertę w kolejce /admin/moderation/listings/; sprawdza zawartość
Admin	Zatwierdza → status: PENDING → APPROVED; oferta pojawia się w wyszukiwarce
Admin	Odrzuca → status: PENDING → REJECTED + komentarz; host jest powiadamiany
Host	Po odrzuceniu: edytuje ofertę; ponownie wysyła do moderacji

13.4 Przepływ ślepego ujawnienia recenzji (Blind Release)

13.4 Przepływ ślepego ujawnienia recenzji (Blind Release)

Gość	Po pobycie wystawia recenzję (reviewer_role=GUEST, is_public=False)
Host	Niezależnie wystawia recenzję gościa (reviewer_role=HOST, is_public=False)
Celery	release_blind_review_task: obie recenzje istnieją? → atomowe ujawnienie (is_public=True)
Celery	Jeśli minie blind_release_at i jedna strona nie oceniła → ujawnienie istniejącej recenzji
Host	Może jednorazowo odpowiedzieć (PATCH /reviews/{uuid}/host-response/)

13.5 Przepływ wyszukiwania AI

13.5 Przepływ wyszukiwania AI	
Użytkownik	Wpisuje zapytanie po polsku, np. 'domek dla 4 na Mazurach z sauną'
Frontend	POST /ai/search/ → tworzy nową AiTravelSession (TTL 24h)
Backend	Pierwszy call do LLM (OpenAI/Groq) → AiFilterInterpretation (lokalizacja, daty, tryb, amenities)
Backend	SearchOrchestrator z AiFilterInterpretation → lista ofert (bez cache Redis)
Backend	Drugi call do LLM → AiRecommendation dla każdej oferty (wyjaśnienie po polsku)
Frontend	Wyświetla karty ofert z wyjaśnieniami 'Ten domek spełnia...' i listą wyróżników
Użytkownik	Może zadać pytanie uzupełniające → POST /ai/search/{session_id}/prompt/

15. Spełnienie minimalnych wymagań projektu

Poniżej szczegółowa analiza spełnienia każdego z czterech minimalnych wymagań stawianych projektowi Django, z dokładnymi wskazaniami na pliki i fragmenty kodu w repozytorium.

14.1 Rejestracja i uwierzytelnianie — django.contrib.auth

Kryterium	Realizacja w projekcie
Użycie django.contrib.auth	Tak — model User dziedziczy po AbstractBaseUser i PermissionsMixin z django.contrib.auth.models. Konfiguracja: AUTH_USER_MODEL = 'users.User' w settings/base.py. Django auth jest w INSTALLED_APPS.
Plik modelu User	backend/apps/users/models.py — klasa User(AbstractBaseUser, PermissionsMixin) z UUID jako PK, email jako USERNAME_FIELD, flagami is_host / is_admin / is_staff / is_active
Rejestracja e-mail/hasło	POST /auth/register/ — serializer waliduje e-mail, hasło (walidatory Django), tworzy użytkownika przez UserManager.create_user(); hasło hashowane PBKDF2 SHA256
Logowanie i JWT	POST /auth/token/ — SimpleJWT weryfikuje credentials, zwraca access token (60 min, HTTP-only cookie) + refresh (30 dni z rotacją). POST /auth/token/refresh/ odświeża access.
Google OAuth 2.0	POST /auth/google/ — token Google weryfikowany przez google-auth; auto-tworzenie konta przy pierwszym logowaniu lub powiązanie z istniejącym przez e-mail. Rejestrowane w AuditLog.
Walidatory hasła	settings/base.py — 4 walidatory Django: UserAttributeSimilarityValidator, MinimumLengthValidator, CommonPasswordValidator, NumericPasswordValidator
Autoryzacja endpointów	DRF permission classes: IsAuthenticated, IsAuthenticatedOrReadOnly, IsHost (custom), IsAdmin (custom) — per endpoint w każdym APIView

Ocena: SPEŁNIONE W CAŁOŚCI

Projekt w pełni wykorzystuje wbudowany system django.contrib.auth poprzez AbstractBaseUser. Implementacja obejmuje rejestrację email/hasło, logowanie JWT, OAuth Google, rotację tokenów refresh i walidatory hasła Django. System uprawnień jest rozbudowany o własne klasy permission.

14.2 Rozbudowany panel admina

Panel Django Admin jest kompleksowo skonfigurowany dla wszystkich 12 aplikacji domenowych. Poniżej zestawienie plików admin.py i ich funkcji:

Plik admin.py	Model	search_fields	list_filter	inlines
listings/admin.py	ListingAdmin	✓ title, slug, email hosta	✓ status, booking_mode, currency	✓ LocationInline + ImageInline
bookings/admin.py	BookingAdmin	✓ email gościa, tytuł oferty	✓ status, currency	✓ StatusHistoryInline

users/admin.py	UserAdmin	✓ email, first_name, last_name, phone	✓ is_host, is_admin, is_staff, is_active	—
messaging/ admin.py	ConversationAdmin	—	—	✓ MessageInline (TabularInline)
reviews/admin.py	ReviewAdmin	✓ listing__title, content	✓ reviewer_role, is_public	—
ai_assistant/ admin.py	AiTravelSessionAdmin	✓ user__email	✓ status	—
pricing/admin.py	SeasonalPricingAdmin	✓	✓	—
discovery/admin.py	DiscoveryAdmin	✓	✓	—

Własne metody list_display — przykłady

- ListingAdmin.location_short() — metoda z @admin.display(description='Miasto', ordering='location__city') zwracająca miasto oferty z obsługą DoesNotExist
- ListingAdmin.image_count() — metoda z @admin.display(description='Zdjęcia', ordering='_image_count') zwracająca liczbę zdjęć (z annotate Count)
- ListingAdmin.get_queryset() — select_related + annotate dla optymalizacji zapytań (N+1 prevention)
- UserAdmin.fieldsets — rozbudowane grupowanie pól: Dane, Uprawnienia, Daty z readonly_fields dla systemowych pól

Inlines — pełna lista

- ListingLocationInline (StackedInline) — edycja lokalizacji bezpośrednio na karcie oferty
- ListingImageInline (TabularInline) — zarządzanie zdjęciami oferty w jednym widoku
- BookingStatusHistoryInline (TabularInline) — historia zmian statusu rezerwacji, readonly
- MessageInline (TabularInline) — wiadomości konwersacji widoczne na karcie Conversation

Ocena: SPEŁNIONE W CAŁOŚCI

Panel admina jest rozbudowany we wszystkich wymaganych obszarach: search_fields i list_filter dla co najmniej 6 modeli, inlines w 4 modelach (Location, Image, StatusHistory, Message), własne metody list_display z @admin.display dekoratorem, fieldsets z grupowaniem pól, raw_id_fields dla wydajności, oraz get_queryset() z annotate dla optymalizacji zapytań.

14.3 Generowanie danych testowych — Faker + komenda seed_db

Kryterium	Realizacja w projekcie
Biblioteka Faker	Faker==30.8.1 w requirements/development.txt; factory-boy==3.3.1 jako dodatkowe narzędzie do tworzenia obiektów modelowych
Komenda manage.py seed_db	backend/apps/common/management/commands/seed_db.py — uruchamia python manage.py seed_db; tworzy użytkowników, oferty, rezerwacje, recenzje, blokady kalendarza z realistycznymi danymi demo
Komenda manage.py seed_polska	backend/apps/common/management/commands/seed_polska.py — specjalistyczne dane dla polskiego rynku: lokalizacje geo (PointField

	PostGIS), polskie opisy z Faker('pl_PL'), opisy regionów
Komenda manage.py seed_mass_listings	backend/apps/common/management/commands/seed_mass_listings.py — masowe generowanie ofert dla testów wydajności wyszukiwania i mapy
Użycie Faker w seed_polska	Faker('pl_PL') generuje polskojęzyczne opisy (paragraph(nb_sentences=3-5)), dane gospodary, opisy ofert per region i typ noclegu
Skrót Makefile	make seed — uruchamia python manage.py seed_db; make seed_polska — dane polskie; udokumentowane w README.md
Dane seed_polska_geo_data.py	backend/apps/common/seed_polska_geo_data.py — statyczne dane geograficzne: regiony Polski, lokalizacje GPS, typy noclegów, tagi geo — używane przez komendy seed

Ocena: SPEŁNIONE W CAŁOŚCI

Projekt posiada trzy osobne komendy management do generowania danych testowych (seed_db, seed_polska, seed_mass_listings). Biblioteki Faker 30.8.1 i factory-boy 3.3.1 są w requirements/development.txt. Faker jest używany z lokalizacją polską ('pl_PL') do generowania opisów. Komendy są udokumentowane w README.md i dostępne przez Makefile.

14.4 Testy jednostkowe

Kryterium	Realizacja w projekcie
Liczba plików testowych	25 plików test_*.py rozmieszczonych w modułach domenowych: users, listings, bookings, pricing, search, discovery, reviews, ai_assistant, common
Framework testowy	pytest 8.3.3 + pytest-django 4.9.0; konfiguracja w pytest.ini; dekorator @pytest.mark.django_db dla testów wymagających bazy
Testy modeli	Testy tworzenia obiektów Booking, User, Listing; weryfikacja soft delete (deleted_at); testy UUID jako PK; testy relacji (Booking → Listing → Host)
Testy serwisów	TestPricingService — test_basic_three_nights, test_extra_adults_and_children_surcharge, test_long_stay_discount, test_polish_holidays_multiplier; TestBookingService — konflikty, anulowania, AvailabilityService.ranges_overlap()
Testy endpointów (HTTP 200)	TestClient DRF dla /auth/register/, /auth/token/, /listings/search/, /bookings/quote/, /bookings/, /conversations/, /ai/search/ — weryfikacja statusów HTTP i struktur odpowiedzi JSON
Fixtures (conftest.py)	backend/conftest.py — fixture listing (oferta APPROVED), approved_listing, guest_user, host_user, host_profile — wielokrotnie używane w testach integracyjnych
Pokrycie kodu (--cov)	CI uruchamia pytest -q --cov; raport pokrycia generowany automatycznie; pipeline blokuje merge przy błędach testów

Ocena: SPEŁNIONE W CAŁOŚCI

Projekt posiada 25 plików testowych pokrywających modele (tworzenie, relacje, soft delete), widoki (kody HTTP, struktury JSON), serwisy biznesowe (PricingService z polskimi świętami, BookingService z konfliktami) i zadania Celery. Testy są uruchamiane automatycznie w CI/CD i blokują merge przy niepowodzeniu.

Podsumowanie spełnienia wymagań

Wymaganie	Status	Lokalizacja w projekcie
Rejestracja i uwierzytelnianie (django.contrib.auth)	✓ **SPEŁNIONE**	apps/users/models.py, apps/users/views.py, config/settings/base.py
Panel admina (search_fields, list_filter, inlines, list_display)	✓ **SPEŁNIONE**	apps/*/admin.py — 11 plików z pełną konfiguracją
Generowanie danych testowych (Faker + komenda seed_db)	✓ **SPEŁNIONE**	apps/common/management/commands/seed_db.py, seed_polska.py, seed_mass_listings.py
Testy jednostkowe (modele, widoki, klucze funkcjonalności)	✓ **SPEŁNIONE**	25 plików test_*.py w apps/*/tests/, conftest.py, pytest.ini

16. Otwarte tematy i dalszy rozwój

Projekt jest funkcjonalnie kompletny — wszystkie kluczowe funkcjonalności są zrealizowane i przetestowane. Poniżej obszary wymagające decyzji biznesowej lub konfiguracji przed pełnym uruchomieniem produkcyjnym.

Temat	Priorytet	Opis i zalecenie
Płatności online (Stripe)	**WYSOKI**	Infrastruktura gotowa (modele Payment, StripeWebhookEvent, idempotency keys). Wymaga konfiguracji konta Stripe, webhook URL i STRIPE_SECRET_KEY. Decyzja: capture po pobycie vs. przy rezerwacji.
Konfiguracja e-mail produkcyjnego	**WYSOKI**	Bez SMTP e-maile nie są wysyłane. Zalecane: Gmail App Password lub SendGrid. Szczegóły w docs/EMAIL_AND_OPS.md.
Konfiguracja Sentry DSN	**ŚREDNI**	Bez DSN błędy produkcyjne nie są monitorowane. Konfiguracja zajmuje 15 minut — warto ustawić przed wdrożeniem.
Testy obciążeniowe	**ŚREDNI**	Zalecane przed wdrożeniem — szczególnie wyszukiwanie geo (PostGIS) i WebSocket (dużo połączeń równoległych).
Moduł wypłat dla hosta	**ŚREDNI**	Endpoint /host/earnings/ istnieje; logika wypłat (Stripe Connect lub przelew tradycyjny) jest do zaprojektowania.
Izochrony (OpenRouteService)	NISKI	Wyszukiwanie ofert do X minut drogi od punktu — nowy moduł z integracją ORS API.
Powiadomienia push (mobile)	NISKI	Czat działa przez WebSocket; push notifications dla użytkownika w tle wymagają integracji FCM/APNs.
Analityka użytkowania	NISKI	Brak wbudowanej analityki; można podpiąć Google Analytics, Plausible lub Mixpanel na poziomie Next.js.

Rekomendacja przed uruchomieniem produkcyjnym

Krok 1 (2h): Skonfigurować SMTP (Gmail App Password lub SendGrid) i Sentry DSN. Krok 2 (1 dzień): Przeprowadzić testy obciążeniowe wyszukiwania PostGIS i WebSocket. Krok 3 (faza beta): Uruchomić płatności Stripe z ograniczoną liczbą użytkowników do walidacji modelu biznesowego.

17. Planowane funkcjonalności — Roadmap

Pięć strategicznych funkcjonalności do wdrożenia w kolejnych etapach rozwoju. Każda wzmacnia pozycję StayMap Polska jako nowoczesnego, danych-centralnego produktu turystycznego.

17.1 AI „Kiedy jechać?" — optymalny termin dla Twoich preferencji

✨ **Inteligentny doradca terminu wyjazdu** **AI · WYSOKI PRIORYTET**

Użytkownik opisuje czego szuka naturalnym językiem — AI analizuje dane sezonowe z SeasonalPricingRule i sugeruje najlepszy miesiąc z uzasadnieniem i prognozą ceny.

- → Użytkownik wpisuje: Chcę ciszy, braku tłumów, dobrej pogody i niskich cen na Mazurach
- → LLM analizuje preferencje → wydobywa parametry: cisza=True, brak_tłumu=True, max_cena=niższy_kwartyl
- → Backend odpytuje SeasonalPricingRule dla regionu — wyciąga historię mnożników per miesiąc
- → AI nakłada filtry: miesiące poza szczytem, niskie mnożniki, dane pogodowe z zewnętrznego API
- → Odpowiedź: Najlepszy termin to wrzesień — ceny o 25% niższe niż w lipcu, jeszcze ciepło, mniej turystów
- → Frontend pokazuje calendar heatmap z kolorami cen per miesiąc i wyjaśnieniem AI
- → Integracja z istniejącym modułem AiTravelSession — rozszerzenie bez budowania od zera

17.2 „W X godzin" — wyszukiwanie izochron od punktu startowego

◎ **Mapa dojazdu zamiast mapy geograficznej** **GEO · WYSOKI PRIORYTET**

Użytkownik wpisuje miasto i czas dojazdu — mapa pokazuje tylko oferty w zasięgu. Myślenie podróżnicze zamiast geograficznego.

- → Użytkownik wpisuje: Warszawa jako punkt startowy, wybiera max 3 godziny jazdy
- → Backend wywołuje OpenRouteService API — generuje izochronę (wielokąt dojazdu) dla 3h jazdy
- → PostGIS ST_Within() filtruje oferty których PointField leży wewnątrz izochrony
- → Leaflet renderuje izochronę jako gradient na mapie + markery ofert w zasięgu
- → Tryby transportu: samochód, rower, kolej — dzięki ORS multi-modal routing
- → Możliwość wyszukiwania z mojej lokalizacji — geolokalizacja przeglądarki jako punkt startowy
- → Infrastruktura PostGIS jest już gotowa — wystarczy nowy endpoint i overlay Leaflet

17.3 „Pamiętnik z podróży" — zdjęcia gości po pobycie

▣ **Autentyczne zdjęcia od gości** **SOCIAL · ŚREDNI PRIORYTET**

Po zakończeniu rezerwacji gość może dodać zdjęcia z pobytu — autentyczne, bez retuszu. Mechanizm jak TripAdvisor dla polskiego rynku.

- → Po COMPLETED — gość otrzymuje e-mail z zaproszeniem do dodania zdjęć
- → Nowy model GuestPhoto (relacja do Booking) z polami: image, caption, taken_at, is_approved
- → Upload przez istniejący mechanizm (Pillow + python-magic) z walidacją MIME
- → Karta oferty: sekcja Zdjęcia Gości poniżej galerii głównej hosta — wyraźnie oznaczona
- → Moderacja opcjonalna — gospodarz może zgłosić zdjęcie do usunięcia (naruszenie prywatności)
- → Wpływ na konwersję: autentyczne zdjęcia budują zaufanie bardziej niż profesjonalna sesja fotograficzna
- → Integracja z Blind Release — zdjęcia widoczne dopiero po zakończeniu okresu recenzji

17.4 Karta podarunkowa „Zarezerwuj komś nocleg”

◆ **Voucher prezentowy z integracją Stripe** **BIZNES · ŚREDNI PRIORYTET**

Kup voucher dla kogoś — wyślij e-mailem lub wydrukuj. Odbiorca wybiera termin i rezerwuje. Idealny prezent urodzinowy, ślubny, na Mikołaja — Stripe już jest w projekcie.

- → Nowy model GiftCard: uuid, value_pln, issued_to_email, redeemed_by_booking, expiry_date, stripe_payment_id
- → Nadawca kupuje voucher przez Stripe — płacąc wartość nominalną (np. 500 zł)
- → Odbiorca dostaje unikalny link /gift/{uuid} — przegląda oferty i aplikuje voucher przy rezerwacji
- → Widget rezerwacyjny: pole Kod podarunkowy — system odlicza wartość od total, nadpłata przez Stripe
- → PDF do wydruku z estetycznym designem: nazwa platformy, wartość, data ważności, kod QR
- → Okazje: urodziny, ślub, rocznica, Mikołaj, Boże Narodzenie — personalizowany komunikat od nadawcy
- → Dodatkowy kanał przychodu: prowizja od wartości vouchera pobierana przy zakupie

17.5 „Mapa wspomnień” — interaktywna historia podróży użytkownika

● **Automatyczny travel journal na profilu** **UX · NISKI PRIORYTET**

Na profilu użytkownika interaktywna mapa Polski z pinezkami wszystkich miejsc gdzie mieszkał przez StayMap. Travel journal, który tworzy się automatycznie.

- → Dane są już w systemie: Booking.listing.location.coordinates + daty + ocena — zero nowego backendu
- → Frontend: nowa zakładka Moje Podróże na /account z Leaflet mapą pełnoekranową
- → Każda pinezka: miniatura zdjęcia oferty, nazwa miejsca, daty pobytu, wystawiona ocena
- → Po kliknięciu pinu — popup z kartą: zdjęcie, adres, Zarezerwowałem X nocy, link do oferty
- → Statystyki na górze mapy: liczba podróży, łączna liczba nocy, odwiedzone województwa
- → Opcja udostępnienia: publiczny link do profilu podróży użytkownika (opt-in)
- → Gamifikacja: odznaki Zwiedzacz za 5 różnych regionów, Maraton za 10 rezerwacji

18. StayMap Polska — szanse na sukces

Kompleksowa ocena pozycji rynkowej platformy, jej wyróżników konkurencyjnych i potencjału wzrostu na polskim rynku turystyki noclegowej.

18.1 Luka rynkowa — dlaczego teraz i dlaczego Polska

Polski rynek turystyki krajowej przeżywa systematyczny wzrost — Polacy realizują rocznie ponad 50 milionów podróży krajowych. Tymczasem dostępne platformy to albo globalne agregatory (Booking.com, Airbnb) niedostosowane do polskiej specyfiki, albo przestarzałe lokalne serwisy bez nowoczesnego UX. StayMap Polska celuje precyzyjnie w tę lukę.

CO BRAKUJE NA RYNKU

- Booking.com nie rozumie polskich świąt i sezonowości
- Airbnb nie ma polskich typów noclegów (agroturystyka, leśniczymówka)
- żadna platforma nie ma mapy jako głównego interfejsu
- Brak myślenia z Warszawy na weekend w 3h jazdy
- Prowizje globalnych graczy (15–20%) zamiast modelu lokalnego

CO STAYMAP DOSTARCZA

- Polski kalendarz świąt wbudowany w silnik cen
- Kategorie i tagi specyficzne dla polskiej turystyki
- Map-first UX jako centralny interfejs odkrywania
- Izochrony W X godzin w roadmapie (ORS API)
- Konfigurowalna prowizja (domyślnie 15%, niższa niż u globalnych graczy)

18.2 Wyróżniki technologiczne

StayMap Polska wyróżnia się na tle konkurencji nie tylko zakresem funkcji, ale jakością architektury — każda decyzja techniczna ma uzasadnienie biznesowe.

Silnik cen uwzględniający polską specyfikę

PricingService jest prawdopodobnie najbardziej zaawansowanym elementem platformy. Łączy sezonowość, polskie święta ustawowe (GUS), szczyty turystyczne definiowane przez admina, reguły hosta i rabaty długoterminowe. Snapshot ceny zamraża wycenę w momencie rezerwacji — gość zawsze płaci kwotę, którą widział przy wyborze. Globalny gracz tego nie wdroży w 6 miesięcy.

AI rozumiejące potrzeby, nie tylko słowa kluczowe

Podczas gdy większość platform oferuje filtry, StayMap rozumie intencję: cichy dom dla rodziny z psem, nie za daleko od Krakowa, początek października to kompletne zapytanie. AI wydobywa parametry, wyszukuje oferty i tłumaczy dopasowanie po polsku. Modularność (OpenAI/Groq) daje pełną kontrolę nad kosztami operacyjnymi.

Mechanizm Blind Release buduje zaufanie w ekosystemie

Recenzje ślepe to nie ficik — to fundament jakości. Gdy obie strony oceniają niezależnie, recenzje są szczere. Buduje to zaufanie gości (oceny są prawdziwe) i chroni gospodarzy (nie boją się odwetu za uczciwą ocenę gościa). Na polskim rynku, gdzie zaufanie do platform rezerwacyjnych bywa niskie, to wyróżnik pierwszego rzędu.

Inteligencja lokalizacji jako automatyczna wartość dla hosta

Gospodarz nie musi opisywać okolicy — system robi to za niego. POI z OpenStreetMap, Destination Score, tekstowe podsumowanie obszaru. To obniża barierę wejścia dla hosta i podnosi jakość oferty. Gość wie co znajdzie w pobliżu zanim zapyta.

18.3 Model biznesowy i skalowalnąść

Strumień przychodu	Status	Potencjał
Prowizja od rezerwacji (domyślnie 15%)	Gotowe	Główne źródło — skaluje się liniowo z liczbą rezerwacji
Karty podarunkowe (vouchery)	Planowane	Prowizja od wartości vouchera + efekt marketingowy
Wyróżnienie ofert hosta (boost)	Do zaprojektowania	Subskrypcja lub płatność jednorazowa za pierwszą pozycję w wynikach
Plan Premium dla hosta	Do zaprojektowania	Zaawansowana analityka, więcej zdjęć, priorytetowa moderacja
Dane analityczne B2B	Dalszy horyzont	Zanonimizowane dane o sezonowości dla branży turystycznej i samorządów

18.4 Strategia wejścia na rynek

Faza	Zakres	Działania kluczowe
Faza 1	Pilot — 2 regiony	Mazury + Zakopane, 200–500 ofert. Program zero prowizji przez pierwsze 6 miesięcy dla hosta. Walidacja UX z prawdziwymi użytkownikami.
Faza 2	Rozszerzenie PL	Dolny Śląsk, Bałtyk, Podlasie, Roztocze. Aktywacja Stripe i płatności. Kampania influencer travel.
Faza 3	Pełna skala	Pełne pokrycie Polski. Wdrożenie Roadmap (izochrony, vouchery, Mapa Wspomnień). B2B: pakiety dla agencji turystycznych.

18.5 Dlaczego StayMap wygra

GŁĘBOKOŚĆ

Lokalnego kontekstu

Polskie święta, polskie regiony, polskie kategorie noclegów, język naturalny po polsku — tego globalny gracz nie wdroży w 6 miesięcy.

JAKOŚĆ

Architektury i danych

PostGIS, WebSocket, Celery, Blind Release, AI — to nie MVP, to produkcyjna platforma. Techdług jest minimalny od dnia zerowego.

ZAUFANIE

Obu stron transakcji

Moderacja, Blind Release, snapshot ceny, AuditLog — każdy mechanizm został zaprojektowany tak, żeby żadna ze stron nie czuła się oszukana.

StayMap Polska to nie kolejny klon Booking.com.

To pierwsza platforma noclegowa zaprojektowana od podstaw dla polskiego podróżnego —

z mapą w centrum, inteligencją AI, uczciwym systemem recenzji i silnikiem cen, który zna Wszystkich Świętych i ferie zimowe.

Infrastruktura jest gotowa. Czas na rynek.

19. Słowniczek pojęć

Termin	Definicja
ASGI	Asynchronous Server Gateway Interface — standard serwera Python obsługujący jednocześnie HTTP i WebSocket; wdrożony przez Daphne
BFF	Backend-for-Frontend — cienka warstwa w Next.js proxy'ująca żądania do Django; upraszcza SSR i konfigurację CORS
Celery	Biblioteka Python do asynchronicznych i cyklicznych zadań (wysyłka e-maili, czyszczenie danych); broker: Redis
Celery Beat	Scheduler Celery do zadań cyklicznych (odpowiednik crona); musi być singletonem w produkcji
CI/CD	Continuous Integration / Continuous Deployment — automatyczne testy i wdrożenia po każdej zmianie kodu
Django Channels	Rozszerzenie Django dodające obsługę WebSocket; używane do czatu w czasie rzeczywistym
GeoJSON	Format danych geograficznych oparty na JSON; używany do granic Polski i województw na mapie frontu
GeoDjango / PostGIS	Rozszerzenia Django i PostgreSQL do geograficznych zapytań; umożliwiają filtrowanie po odległości
JWT	JSON Web Token — token dostępu bez sesji serwerowej; access (60 min, HTTP-only cookie) + refresh (30 dni z rotacją)
LLM	Large Language Model — duży model językowy (np. GPT-4o-mini); używany w module AI do interpretacji zapytań
Nominatim	Silnik geokodowania OpenStreetMap; zamienia nazwę miejsca na współrzędne GPS
OpenAI-compatible API	Interfejs API kompatybilny z OpenAI; pozwala użyć alternatywnych dostawców (Groq z Llama) bez zmiany kodu
Overpass API	API do odpytywania danych OpenStreetMap o punkty zainteresowania (POI) w zadanym obszarze
POI	Point of Interest — punkt zainteresowania (restauracja, muzeum, szlak) używany do oceny atrakcyjności okolicy
Redis	Baza klucz-wartość w pamięci; w projekcie: cache zapytań, broker Celery, warstwa kanałów WebSocket
REST API	Architektura API oparta na HTTP; endpointy zwracają JSON; używane przez frontend do komunikacji z backendem
Sentry	Platforma monitorowania błędów; przechwytuje wyjątki produkcyjne z pełnym kontekstem i stack trace
Soft delete	Oznaczenie rekordu jako usunięty (deleted_at) zamiast fizycznego kasowania; umożliwia przywrócenie
SSR	Server-Side Rendering — renderowanie stron HTML na serwerze Next.js; lepsze SEO i czas pierwszego ładowania
Stripe	Płatność online; infrastruktura (modele, idempotency keys) przygotowana, oczekuje konfiguracji biznesowej
Throttling	Ograniczenie liczby zapytań do API w jednostce czasu; ochrona przed brute-force i nadużyciami

TTL	Time to Live — czas życia rekordu w cache; po upływie TTL dane są uznawane za przestarzałe
UUID	Universally Unique Identifier — globalnie unikalny identyfikator; klucz główny wszystkich modeli w projekcie
WebSocket	Protokół komunikacji dwukierunkowej w czasie rzeczywistym; używany w czacie między gościem a gospodarzem